



세미나

8.4

Using pre-trained word embeddings

정보통계학과 이현섭

8.2의 모델과 다른 접근 방법으로 분석

- 8.2장에서의 모델들은 모델이 학습한 단어 시퀀스로부터 벡터를 만들기 위해 임베딩 레이어를 포함!
- 이번 강에서는 임베딩 레이어를 임의의 값으로 시작하여 다른 파라미터와 함께 학습하는 대신 사전 훈련된 임베딩을 제공!
(책 저자는 이러한 접근 방식이 가능하며 어떻게 시작할 수 있는지를 알려줌)
- 사용할 데이터 : glove.6B.50d.txt
(원래대로면 GloVe 6B를 전체적으로 불러오지만, 데이터 다운로드가 되지 않아 일부만 txt 파일로 불러 진행)

필요한 라이브러리

- **tidyverse** : 데이터 분석을 위한 포괄적인 프레임워크. 데이터 랭글링(자료를 쉽게 접근할 수 있도록 데이터를 정리하고 통합하는 과정), 시각화, 모델링 등을 위한 다양한 기능 제공.
- **tidymodels** : tidyverse 프레임워크 기반의 머신러닝 패키지. 모델 학습, 평가, 배포 등을 위한 일관된 인터페이스 제공.
- **textrecipes** : 텍스트 데이터 전처리 및 분석을 위한 패키지. 텍스트 정제, 토큰화, 특징 추출 등을 위한 다양한 기능 제공.
- **keras** : 딥러닝 모델 개발을 위한 고수준 API. TensorFlow 백엔드 기반으로 다양한 딥러닝 모델 구축 및 학습 가능.
- **tensorflow** : 딥러닝 및 머신러닝을 위한 오픈소스 프레임워크. 다양한 딥러닝 모델 구축 및 학습, 배포 가능.
- **stopwords** : 자연어 처리에서 일반적으로 제거되는 불필요한 단어 목록 제공.
- **textdata** : 텍스트 데이터 분석을 위한 다양한 유틸리티 함수 제공.
- **tibble** : 데이터 프레임을 위한 tidyverse 표준 형식. 데이터를 행과 열로 구성된 표 형식으로 표현.
- **rsample** : R 언어에서 데이터 분석을 위한 리샘플링 및 교차 검증을 수행하는 패키지

이전의 R 데이터 불러오기

```
install.packages("tidyverse")  
library(tidyverse)
```

```
kickstarter <- read_csv("C:/Users/user/Desktop/2024-1/R 세미나/kickstarter.csv")  
kickstarter
```

- 앞으로 있을 glove.txt와 함께 사용할 kickstarter.csv파일을 불러오기

```
#tidymodels 설치  
#install.packages("tidymodels")  
library(tidymodels)
```

```
#textrecipes 설치  
#install.packages("textrecipes")  
library(textrecipes)
```

- 데이터 사용을 위해 라이브러리 패키지 설치

Kickstarter 데이터 확인해보기

```
# A tibble: 269,790 × 3
  blurb                                state created_at
  <chr>                                <dbl> <date>
1 Exploring paint and its place in a digital world. 0 2015-03-17
2 Mike Fassio wants a side-by-side photo of me and Hazel eating cake with our bare hands. Let's make this a reality! 0 2014-07-11
3 I need your help to get a nice graphics tablet and Photoshop! 0 2014-07-30
4 I want to create a Nature Photograph Series of photos of wildlife in Sacramento and surrounding areas. 0 2015-05-08
5 I want to bring colour to the world in my own artistic skills. Going abroad to north Africa to share my experience and art... 0 2015-02-01
6 We start from some lovely pictures made by us and we decide to continue with our dream! 0 2015-11-18
7 Help me raise money to get a drawing tablet 0 2015-04-03
8 I would like to share my art with the world and to do that I must print it to a physical medium. http://solacenigma420.de... 0 2014-10-15
9 Post Card don't set out to simply decorate stories. Our goal is to combine visual imagery and language to enhance the sto... 0 2015-06-25
10 My name is Siu Lon Liu and I am an illustrator seeking funds to market my start up, Studio MyMu. 0 2014-07-19
# i 269,780 more rows
# i Use `print(n = ...)` to see more rows
```

- blurb : 캠페인에 대한 설명
- state : 캠페인이 크라우드딩 펀딩 목표를 달성했는지에 대한 여부
- created_at : 캠페인이 게시된 시간

The screenshot shows the Kickstarter homepage. At the top, there's a search bar and a 'Start a project' button. Below the navigation bar, there are filters for 'Show me' (Music), 'projects on' (Earth), and 'sorted by' (Magic). A 'More filters' link is also present. The main section displays 'Explore 68,879 projects' with a grid of project cards. Each card includes a project image, title, creator name, and funding progress.

Project Title	Creator	Funding Progress
EIGAKAN Jazz Kissa Project	James H Catchpole	9 days left • 53% funded
Christopher Young's "Nosferatu - A Symphony of Horror" CD	Robert Young	2 days left • 89% funded
LET ME HOLD A DOLLA! LARUSSELL GOES MAJORLY INDEPENDENT	LaRussell	1 days left • 3867300% funded

이전의 R 데이터 불러오기

```
#kick_rec 만들기
max_words <- 2e4
max_length <- 30
kick_rec <- recipe(~ blurb, data = kickstarter_train) %>%
  step_tokenize(blurb) %>%
  step_tokenfilter(blurb, max_tokens = max_words) %>%
  step_sequence_onehot(blurb, sequence_length = max_length)

#kick_rec 확인하기
kick_rec
```

- 저번 장에서 사용했던 kick_rec 만들기(있는 경우 다시 확인하기)

— Recipe —

— Inputs

Number of variables by role
predictor: 1

— Operations

- Tokenization for: blurb
- Text filtering for: blurb
- Sequence 1 hot encoding for: blurb

- kick_rec의 정보 확인

- blurb 열이 단일 예측 변수임을 나타내며, tokenization과 text filtering, one-hot encoding을 진행

이전의 R 데이터 불러오기

```
#kick_prep 만들기  
kick_prep <- prep(kick_rec)
```

- kick_prep 만들기(있는 경우 다시 확인하기)

— Recipe

— Inputs

Number of variables by role
predictor: 1

— Training information

Training data contained 202092 data points and no incomplete rows.

— Operations

- Tokenization for: blurb | *Trained*
- Text filtering for: blurb | *Trained*
- Sequence 1 hot encoding for: blurb | *Trained*

- kick_prep의 정보 확인
- prep함수는 데이터 전처리를 위해 사용한 함수
- 함수를 사용하여 trained 되었음을 보여주고 있음

데이터 파일 불러오기(txt)

```
# 다운로드한 파일의 경로를 지정합니다.  
file_path <- "C://Users//user//Desktop//2024-1//R 세미나//glove.6B.50d.txt"  
  
# 파일을 읽어와서 데이터를 로드합니다.  
glove_data <- read.table(file_path, header = FALSE, sep = " ")
```

- txt파일을 불러오고 데이터를 로드

```
#tidymodels 설치  
#install.packages("tidymodels")  
library(tidymodels)  
  
#textrecipes 설치  
#install.packages("textrecipes")  
library(textrecipes)
```

- 데이터 사용을 위해 라이브러리 패키지 설치

데이터프레임을 *tibble*로 변환하여 확인

```
library(tibble)

# 데이터프레임을 tibble로 변환하기.
glove6b_tibble <- as_tibble(glove_data)
```

- txt파일을 불러오고 데이터를 로드
- 만들어진 glove6b_tibble 확인하기
- 임베딩의 어휘 크기는 학습된 레시피가 기대한 것보다 훨씬 크다고 함.

tibble로 된 데이터프레임 확인

```
  v1      v2      v3      v4      v5      v6      v7      v8      v9      v10     v11     v12     v13     v14     v15     v16     v17     v18
  <chr>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 "the"    0.418    0.250   -0.412    0.122    0.345   -0.0445  -0.497  -0.179  -6.60e-4 -0.657    0.278   -0.148   -0.557    0.147  -0.00951  0.0117  0.102
2 ", "    0.0134    0.237   -0.169    0.410    0.638    0.477   -0.429  -0.556  -3.64e-1 -0.239    0.130   -0.0637  -0.396   -0.482    0.233    0.0902  -0.133
3 ". "    0.152    0.302   -0.168    0.177    0.317    0.340   -0.435  -0.311  -4.50e-1 -0.295    0.166    0.120   -0.413   -0.424    0.599    0.288   -0.115
4 "of"     0.709    0.571   -0.472    0.180    0.544    0.726    0.182  -0.524    1.04e-1 -0.176    0.0789  -0.362   -0.118   -0.833    0.119   -0.166    0.0616
5 "to"     0.680   -0.0393    0.302  -0.178    0.430    0.0322  -0.414    0.132  -2.98e-1 -0.0853    0.171    0.224   -0.100   -0.437    0.334    0.678    0.0572
6 "and"    0.268    0.143   -0.279    0.0163    0.114    0.699   -0.513  -0.474  -3.31e-1 -0.138    0.270    0.309   -0.450   -0.413   -0.0993    0.0381  0.0297
7 "in"     0.330    0.250   -0.609    0.109    0.0364    0.151   -0.551  -0.0742  -9.23e-2 -0.328    0.0960  -0.823   -0.367   -0.670    0.429    0.0165  -0.236
8 "a"      0.217    0.465   -0.468    0.101    1.01     0.748   -0.531  -0.263    1.68e-1  0.132   -0.249   -0.442   -0.217    0.510    0.134   -0.431  -0.0312
9 " 0.25... NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA      NA
# i 33 more variables: v19 <dbl>, v20 <dbl>, v21 <dbl>, v22 <dbl>, v23 <dbl>, v24 <dbl>, v25 <dbl>, v26 <dbl>, v27 <dbl>, v28 <dbl>,
#   v29 <dbl>, v30 <dbl>, v31 <dbl>, v32 <dbl>, v33 <dbl>, v34 <dbl>, v35 <dbl>, v36 <dbl>, v37 <dbl>, v38 <dbl>, v39 <dbl>, v40 <dbl>,
#   v41 <dbl>, v42 <dbl>, v43 <dbl>, v44 <dbl>, v45 <dbl>, v46 <dbl>, v47 <dbl>, v48 <dbl>, v49 <dbl>, v50 <dbl>, v51 <dbl>
```

- tibble된 데이터인 glove6b_tibble 확인하기.

kick_prep 데이터 확인

```
> tidy(kick_prep)
# A tibble: 3 x 6
  number operation type      trained skip id
  <int> <chr>      <chr>      <lgl>  <lgl> <chr>
1     1 step      tokenize    TRUE   FALSE tokenize_D80KV
2     2 step      tokenfilter TRUE    FALSE tokenfilter_8G3fR
3     3 step      sequence_onehot TRUE    FALSE sequence_onehot_61zOW
```

- kick_prep 데이터를 tidy한 형태로 정리. (데이터를 tidy 형태로 변환하는 작업을 수행)
- 특징 : 각 변수는 열을 생성, 각 관측치는 행을 형성, 각 관측 단위별로 별도의 테이블이 있음 등

```
> tidy(kick_prep, number = 3)
# A tibble: 20,000 x 4
  terms vocabulary token id
  <chr>      <int> <chr> <chr>
1 blurb         1 0      sequence_onehot_61zOW
2 blurb         2 00     sequence_onehot_61zOW
3 blurb         3 000    sequence_onehot_61zOW
4 blurb         4 00pm   sequence_onehot_61zOW
5 blurb         5 01     sequence_onehot_61zOW
6 blurb         6 02     sequence_onehot_61zOW
7 blurb         7 03     sequence_onehot_61zOW
8 blurb         8 05     sequence_onehot_61zOW
9 blurb         9 07     sequence_onehot_61zOW
10 blurb        10 09     sequence_onehot_61zOW
# i 19,990 more rows
# i Use `print(n = ...)` to see more rows
```

- 각 변수에 대해 최대 3개의 유일한 값을 유지.

모델에 들어갈 변수 및 DNN 모델 만들기

```
# glove6b_matrix 만들기
glove6b_matrix <- tidy(kick_prep, 3) %>%
  select(token) %>% #glove6b_matrix에서 token열만 선택
  left_join(glove6b_tibble, by = "token",) %>% #token열을 기준으로 tidy_glove와 조인한다.(glove 데이터를 추가하기 위해서)
  mutate(across(!token, replace_na, 0)) %>% #token열을 제외한 모든 열에 대해 결측값을 0으로 대체
  select(-token) %>% #token열을 제외한 모든 열을 선택
  as.matrix() %>% #데이터를 행렬 형식으로 변환함.
  rbind(0, .) #행렬의 첫 번째 행에 0 벡터를 추가함.
```

- glove_matrix를 만들기

```
dense_model_pte <- keras_model_sequential() %>% #sequential모델을 생성
  layer_embedding(input_dim = max_words + 1, #input_dim은 max_words+1한 값
                  output_dim = ncol(glove6b_matrix), #output_dim은 glove6b_matrix의 컬럼 값
                  input_length = max_length) %>% #입력 sequence는 max_length의 길이와 같도록
  layer_flatten() %>% #Flatten레이어를 추가(1차원으로 펼쳐줌)
  layer_dense(units = 32, activation = "relu") %>% #units는 뉴런의 수, activation은 활성화 함수를 의미
  layer_dense(units = 1, activation = "sigmoid") #여기서는 activation으로 sigmoid 사용
```

- dense_model_pte를 만들기

DNN 모델의 가중치 및 optimizer 설정

```
dense_model_pte %>%  
  get_layer(index = 1) %>% #첫 번째 레이어에 설정  
  set_weights(list(glove6b_matrix)) %>% #가중치를 고정  
  freeze_weights()  
  
dense_model_pte %>% compile(  
  optimizer = "adam",  
  loss = "binary_crossentropy",  
  metrics = c("accuracy")  
) #compile하여 optimizer, loss, metrics를 설정
```

- dense_model_pte의 형식을 설정하기
- optimizer를 설정하여 DNN 모델에 작동할 변수 설정하기
- 가중치를 동결하려면 freeze_weights() 함수를 사용해야 한다!

glove6b_matrix 데이터 확인

```
> glove6b_matrix
```

	d1	d2	d3	d4	d5	d6	d7	d8	d9
[1,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[2,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[3,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[4,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[5,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[6,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[7,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[8,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[9,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[10,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[11,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000
[12,]	0.000	0.00000	0.000	0.00000	0.00000	0.00000	0.000	0.00000	0.000000

- glove6b_matrix의 형태 및 데이터 확인하기

DNN 모델의 parameter 및 구현 확인

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 30, 50)	1000050
flatten (Flatten)	(None, 1500)	0
dense_1 (Dense)	(None, 32)	48032
dense (Dense)	(None, 1)	33

Total params: 1048115 (4.00 MB)

Trainable params: 48065 (187.75 KB)

Non-trainable params: 1000050 (3.81 MB)

- dense_model_pte 모델의 parameter 개수 및 output shape 확인

Model: "sequential"

Layer (type)	Output Shape	Param #	Trainable
embedding (Embedding)	(None, 30, 50)	1000050	N
flatten (Flatten)	(None, 1500)	0	Y
dense_1 (Dense)	(None, 32)	48032	Y
dense (Dense)	(None, 1)	33	Y

Total params: 1048115 (4.00 MB)

Trainable params: 48065 (187.75 KB)

Non-trainable params: 1000050 (3.81 MB)

- dense_model_pte 모델의 parameter 개수 및 output shape 확인(Trainable Y, N도 확인)

결과 확인 및 해석

Final epoch (plot to see history):

loss: 0.6694

accuracy: 0.5839

val_loss: 0.6848

val_accuracy: 0.5558

- 20epochs를 진행하였을 때의 결과는 위의 사진과 같다
- accuracy는 0.5839, val_accuracy는 0.5558이다.
- 이는 높은 확률이라고 보기 힘들며, 현재 모형에서 조정해 더 나은 값을 찾아야 하는 것을 의미한다.

새로운 모델을 위한 사용자 함수 만들기

```
#keras_predict라는 사용자 함수 만들기
keras_predict <- function(model, baked_data, response) {
  predictions <- predict(model, baked_data)[, 1]
  tibble(
    .pred_1 = predictions,
    .pred_class = if_else(.pred_1 < 0.5, 0, 1),
    state = response
  ) %>%
  mutate(across(c(state, .pred_class),          ## create factors
                 ~ factor(.x, levels = c(1, 0)))) ## with matching levels
}
```

- keras_predict라는 사용자 함수를 설정하여 만들기
- baked_data는 입력 데이터는 나타내고 model은 예측에 사용될 모델이다.
- predict 함수의 결과는 각 샘플에 대한 예측 확률이므로 저장한다.
- tibble 함수를 사용하여 결과는 tibble 형태로 변환한다. (pred 1열에는 예측 확률이 저장, pred_class 열에는 예측된 클래스(0 or 1)가 저장됨.
- across 함수를 사용하여 state열과 pred_class열을 모두 factor로 변환.

사용자 함수를 사용하여 predict 진행

```
> pte_res
# A tibble: 50,524 x 3
  .pred_1 .pred_class state
  <dbl>   <fct>      <fct>
1    0.392 0          0
2    0.558 1          0
3    0.503 1          0
4    0.466 0          0
5    0.382 0          1
6    0.423 0          1
7    0.526 1          0
8    0.446 0          0
9    0.446 0          1
10   0.422 0          1
```

- 만든 함수로 생성 및 생성 결과 확인
- 0.5를 기준으로 구분되는 모습을 볼 수 있음.

사용자 함수를 사용하여 predict 진행

```
# A tibble: 2 x 3
  .metric      .estimator .estimate
  <chr>        <chr>         <dbl>
1 accuracy    binary         0.556
2 kap         binary         0.0972
```

- accuracy 및 kap 확인(정확도와 카파 계수)
- 카파 계수 : 모델의 정확도가 우연의 일치 수준보다 얼마나 더 높은지를 나타내는 지표, 0은 우연의 일치, 1은 완벽한 일치를 의미한다.
- 결과적으로는 55.6%의 정확도와 0.0972의 카파 계수를 보여준다. 우연의 일치 수준에 가깝고, 모델 성능 향상이 더욱 필요하다.

새로운 DNN 모델 구현

```
#딥러닝 모델 구현
dense_model_pte2 <- keras_model_sequential() %>%
  layer_embedding(input_dim = max_words + 1,
                  output_dim = ncol(glove6b_matrix),
                  input_length = max_length) %>%
  layer_flatten() %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dense(units = 1, activation = "sigmoid")
```

```
#형식 설정
dense_model_pte2 %>%
  get_layer(index = 1) %>%
  set_weights(list(glove6b_matrix))
```

```
#optim 설정
dense_model_pte2 %>% compile(
  optimizer = "adam",
  loss = "binary_crossentropy",
  metrics = c("accuracy")
)
```

```
#train 진행
dense_pte2_history <- dense_model_pte2 %>% fit(
  x = kick_analysis,
  y = state_analysis,
  batch_size = 512,
  epochs = 20,|
  validation_data = list(kick_assess, state_assess),
  verbose = FALSE
)
```

```
#evaluation
pte2_res <- keras_predict(dense_model_pte2, kick_assess, state_assess)
metrics(pte2_res, state, .pred_class)
```

- 딥러닝 모델 구현 및 형식과 optimizer 설정

- train 진행 및 eval에 대한 코드

새로운 DNN 모델의 parameter 확인

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 30, 50)	1000050
flatten_2 (Flatten)	(None, 1500)	0
dense_5 (Dense)	(None, 32)	48032
dense_4 (Dense)	(None, 1)	33

Total params: 1048115 (4.00 MB)

Trainable params: 1048115 (4.00 MB)

Non-trainable params: 0 (0.00 Byte)

- 새로 만든 DNN 모델의 Output shape 및 parameter 확인

predict한 결과 및 정확도 확인

```
> pte2_res <- keras_predict(dense_model_pte2, kick_assess, state_assess)
1579/1579 [=====] - 1s 410us/step
> pte2_res
# A tibble: 50,524 x 3
      .pred_1 .pred_class state
      <dbl>   <fct>      <fct>
1 0.000157     0         0
2 0.00152      0         0
3 0.0000277    0         0
4 0.00000802   0         0
5 1            1         1
6 1.00         1         1
7 0.00000000102 0         0
8 0.00169      0         0
9 0.460        0         1
10 1.00         1         1
# i 50,514 more rows
# i Use `print(n = ...) ` to see more rows
```

- 새로운 DNN 모델로 predict를 한 결과

```
> metrics(pte2_res, state, .pred_class)
# A tibble: 2 x 3
  .metric .estimator .estimate
  <chr>   <chr>      <dbl>
1 accuracy binary    0.800
2 kap     binary    0.598
```

- 이에 대한 정확도 및 카파 계수 확인
- 이전보다 더 높은 정확도 및 카파 계수를 기록함에 따라 성능이 더 향상되었음을 볼 수 있다.